

CHAPTER 04

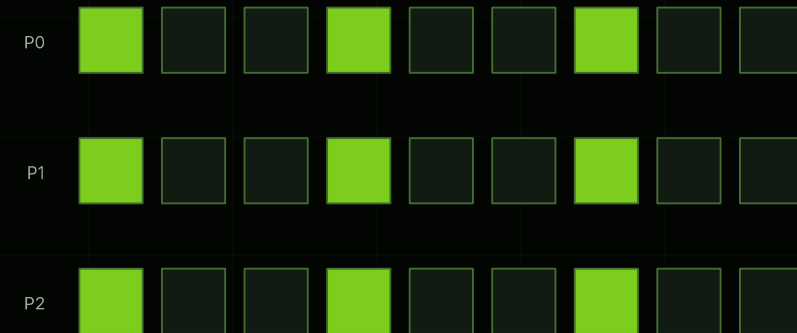
분산 메시지 큐 설계

Topic

Partition

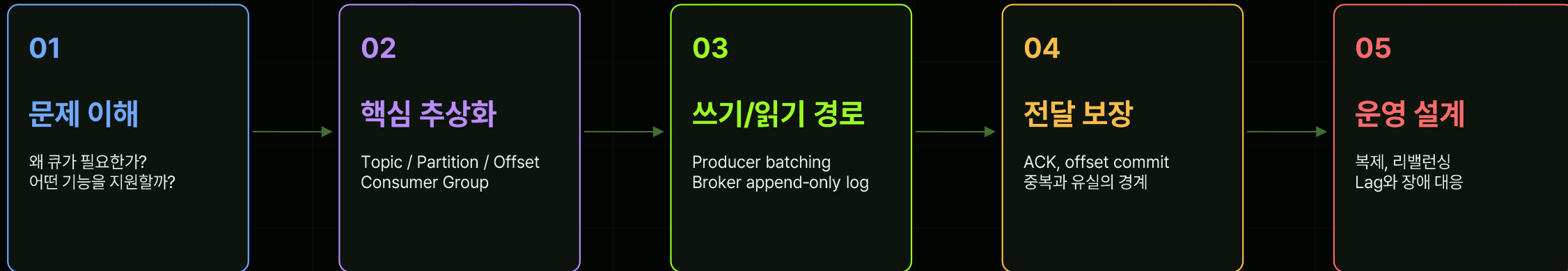
Offset

Consumer Group



오늘의 설계 관점

메시지 큐는 단순한 “중간 저장소”가 아니라, 비동기 시스템의 확장 단위입니다.



문제 이해 및 범위 정의

“분산 메시지 큐”라는 말 안에 worker queue와 event streaming의 요구가 섞여 있습니다.

면접관에게 확인할 질문

- 메시지는 얼마나 오래 보관해야 하나요?
- 같은 메시지를 여러 소비자가 반복 소비해야 하나요?
- 전체 순서가 필요한가요, 아니면 key별 순서면 충분한가요?
- 전달 보장은 at-most / at-least / exactly-once 중 무엇인가요?
- 처리량과 지연 중 무엇을 더 우선하나요?

이번 설계에서 포함

- Producer가 topic에 메시지 발행
- Consumer group이 topic을 구독
- 파티션 단위 저장·복제·순서 보장
- 오프셋 기반 반복 소비와 재처리
- 장애 시 leader 전환과 리밸런싱

이번 설계에서 제외

- 메시지 payload 스키마 레지스트리 상세
- 복잡한 스트림 처리 엔진
- SaaS형 multi-tenant 과금/권한 전체
- 정확히 한 번 처리의 외부 DB 트랜잭션 구현
- 메시지 내용 검색 시스템

요구사항 정리

요구사항은 “기본 메시지 큐 기능”과 “대규모 로그 시스템 기능”으로 나뉘어 있습니다.

기능 요구사항

- 생산자는 topic에 메시지를 발행할 수 있어야 합니다.
- 소비자는 topic을 구독하고 offset부터 읽을 수 있어야 합니다.
- 하나의 topic은 여러 partition으로 나뉘어 확장됩니다.
- 같은 consumer group 안에서는 파티션을 나누어 병렬 소비합니다.
- 메시지는 보관 기간 안에서 다시 읽을 수 있어야 합니다.
- 전달 보장 수준을 선택할 수 있어야 합니다.

비기능 요구사항

- High throughput — 대량 메시지를 batch 단위로 처리합니다.
- Low latency — producer/consumer 응답 시간을 제어합니다.
- Scalability — broker와 partition을 늘려 수평 확장합니다.
- Durability — 장애가 나도 committed message를 잃지 않습니다.
- Availability — 일부 broker 장애에도 읽기/쓰기를 계속합니다.
- Observability — lag, ISR, latency를 추적합니다.

대략적 규모 가정

숫자는 예시입니다. 계산 목적은 “어떤 병목을 먼저 해결해야 하는가”를 보는 것입니다.

100K/s

초당 메시지 수

1KB

평균 메시지 크기

8.64TB

일일 원본 데이터

3x

복제 계수

~181TB

7일 보관량

저장량 계산

$100,000 \text{ msg/s} \times 1\text{KB} \times 86,400\text{s} \approx 8.64\text{TB/day}$
복제 3개 $\times$ 7일 $\approx 181\text{TB}$

처리량 계산

1KB 메시지를 하나씩 보내면 요청 수가 병목입니다.
따라서 producer/broker/consumer 모두 batch 단위로 움직여야 합니다.

데이터가 커지는 속도

1시간

1일

7일×복제

계산이 알려주는 설계 방향

높은 처리량과 장기 보관을 동시에 만족하려면 “큐”보다 “분산 로그”에 가까워집니다.

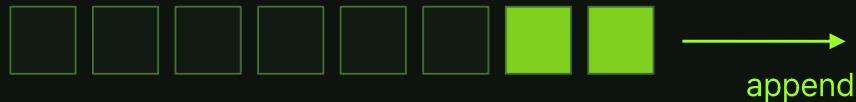
피해야 할 방향: 중앙 DB 큐

messages table

status update
ack/delete

- random write와 row lock 증가
- 소비 상태가 데이터 본문과 뒤섞임
- 반복 소비와 장기 보관이 어려움
- 단일 DB가 처리량 병목이 되기 쉬움

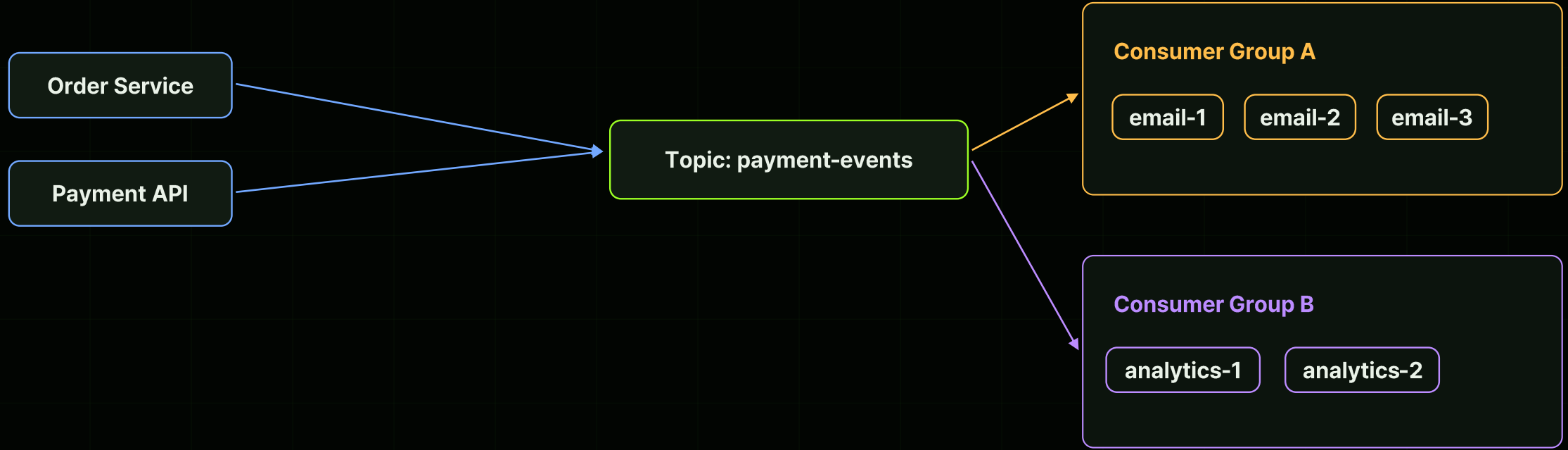
선택할 방향: append-only partition log



- 쓰기: 순차 append
- 읽기: offset 기반 sequential scan
- 소비 상태는 별도 offset으로 관리
- partition을 늘려 병렬성과 저장량을 확장

메시징 모델: Queue와 Pub/Sub를 하나로 보기

Topic과 Consumer Group을 조합하면 두 모델을 모두 표현할 수 있습니다.



Point-to-point처럼 동작

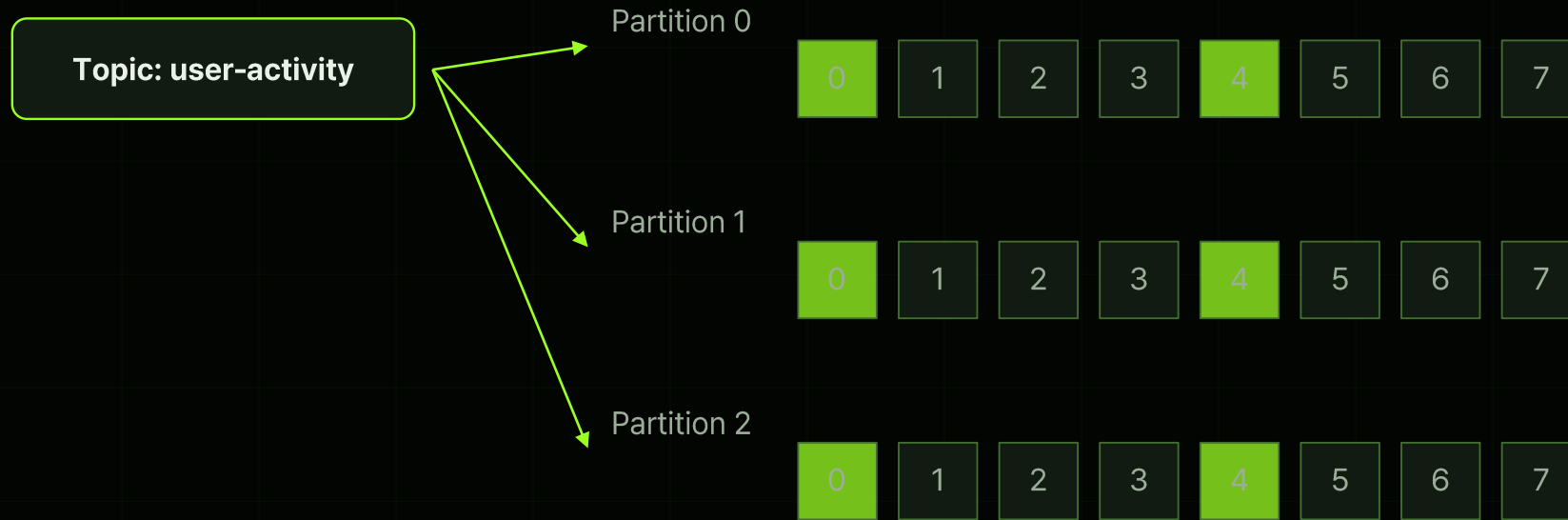
같은 consumer group 안에서는 각 메시지가 하나의 consumer에게만 배정되어 load balancing이 됩니다.

Publish-subscribe처럼 동작

서로 다른 consumer group은 같은 topic을 독립적으로 읽기 때문에 "구독자별 복사본"처럼 동작합니다.

핵심 추상화: Topic, Partition, Offset

순서 보장과 수평 확장의 단위가 서로 다르다는 점이 중요합니다.



Topic

메시지를 분류하는 논리적 이름입니다.

Partition

물리적 저장·복제·순서 보장의 단위입니다.

Offset

파티션 안에서 메시지 위치를 나타냅니다.

API 설계

실제 시스템에서는 REST보다 SDK/프로토콜 호출이 많지만, 면접에서는 역할별 API로 설명하면 깔끔합니다.

Producer

send(topic, key, value, headers)

partition key, timeout, acks, compression, idempotency

Consumer

poll(topic, group_id, offset)

max_bytes, wait_time, partition assignment, fetch offset

Commit

commit(group_id, topic, partition, offset)

처리 완료 지점을 저장하여 재시작 시 이어 읽기

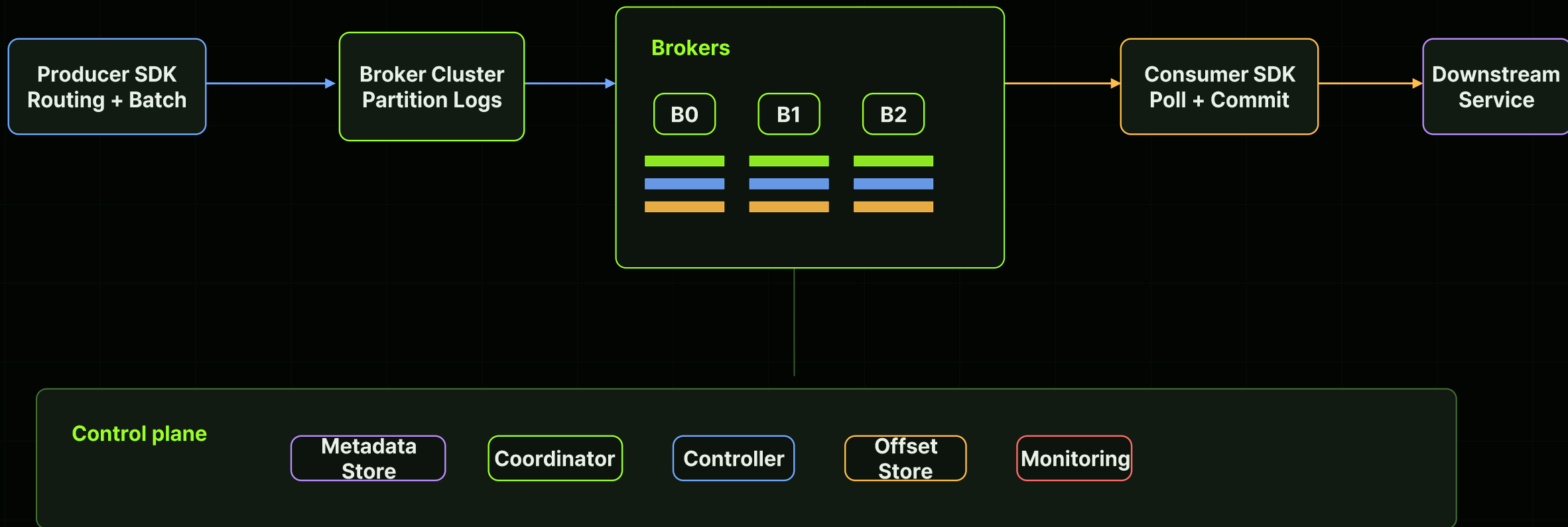
Admin

createTopic(name, partitions, replicas)

retention, cleanup policy, min ISR, quota 설정

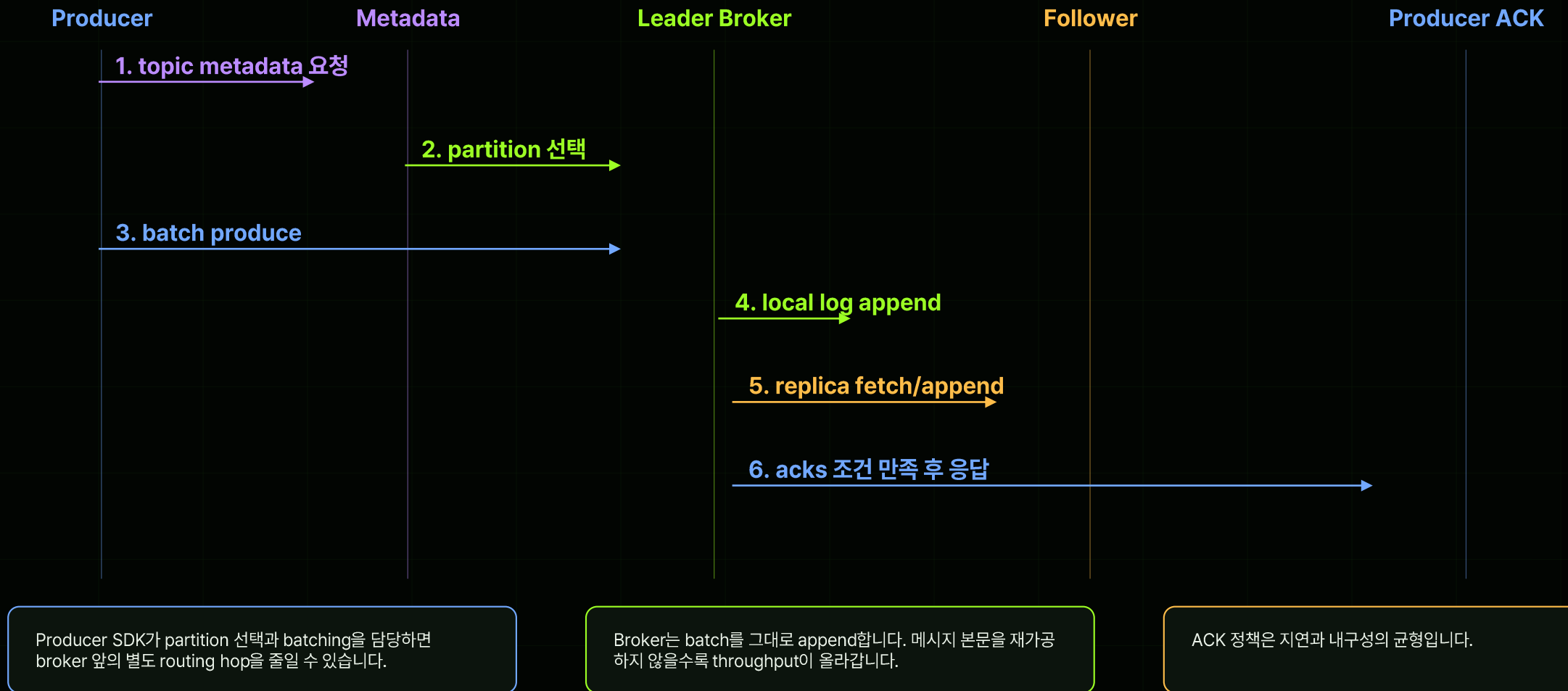
개략적 아키텍처

Data plane은 메시지를 빠르게 이동시키고, Control plane은 배치·리더·상태를 관리합니다.



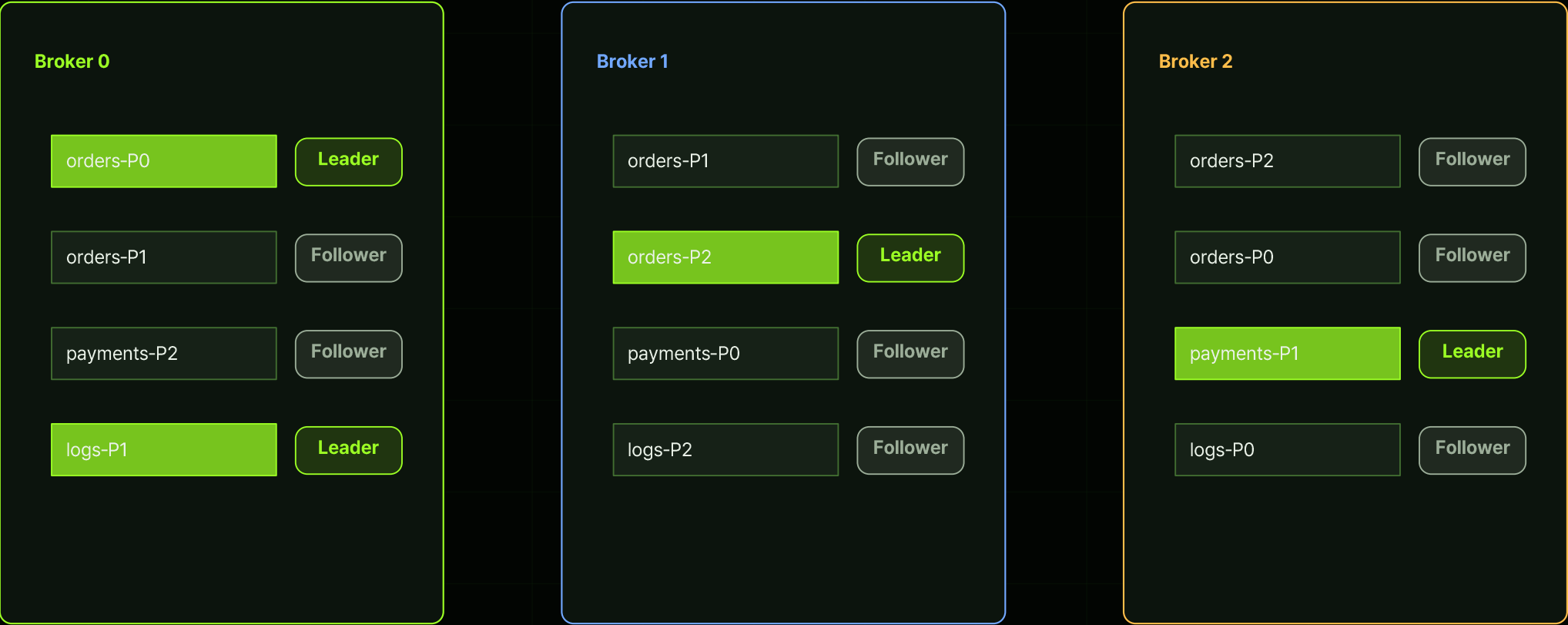
쓰기 경로 상세

Producer는 metadata를 보고 partition을 고른 뒤, batch를 leader broker에 보냅니다.



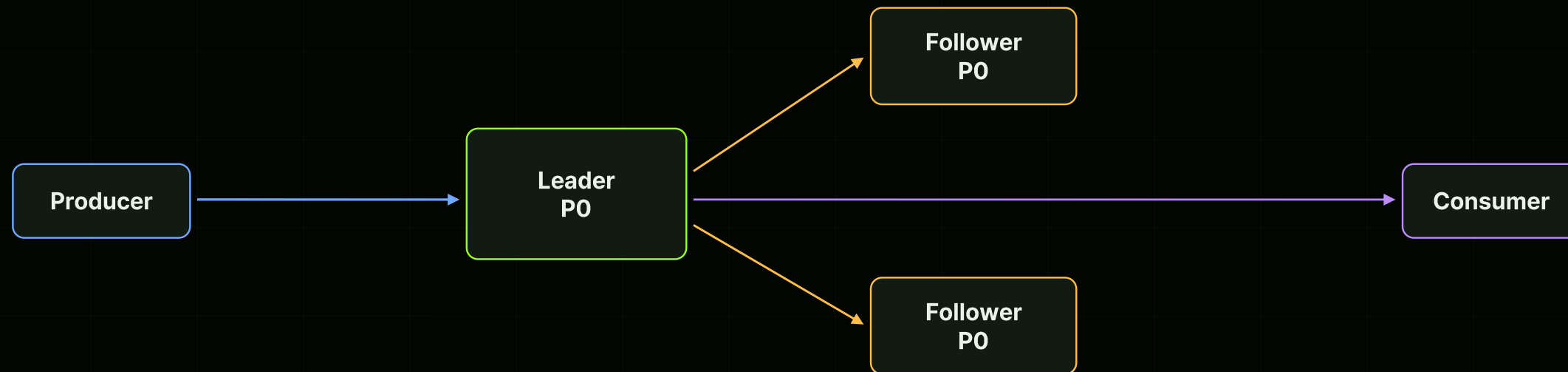
파티션과 브로커 배치

Partition은 저장·복제·병렬 처리의 단위입니다. Broker는 여러 topic의 partition을 함께 보관합니다.



복제와 Leader / Follower

각 partition에는 leader가 있고, follower replica가 leader의 log를 따라옵니다.



Write

Producer는 partition leader에 기록합니다.

Replication

Follower는 leader log를 fetch하여 복제합니다.

ISR

동기화가 충분히 따라온 replica 집합입니다.

메시지 수신 응답 ACK: 지연과 내구성의 스위치

Producer가 언제 “쓰기 성공”으로 간주할지 결정합니다.

acks=0

브로커 응답을 기다리지 않음

Latency : 가장 빠름
Durability : 유실 가능성 높음

acks=1

leader가 local log에 쓰면 성공

Latency : 중간
Durability : leader 장애 직후 유실 가능

acks=all

ISR replica까지 확인 후 성공

Latency : 느림
Durability : 가장 강한 내구성

낮은 지연



높은 내구성

저장소 설계: Append-only Log

메시지는 수정하지 않고 partition log 끝에 붙입니다. 읽기는 offset으로 위치를 찾아갑니다.

Partition P0 on Broker 1

segment-000.log
0~999

segment-001.log
1000~1999

segment-002.log
2000~2999

active segment
3000~

→
append

Log file

message batch가 순서대로
저장됩니다.

Index file

offset → byte position을
빠르게 찾습니다.

Time index

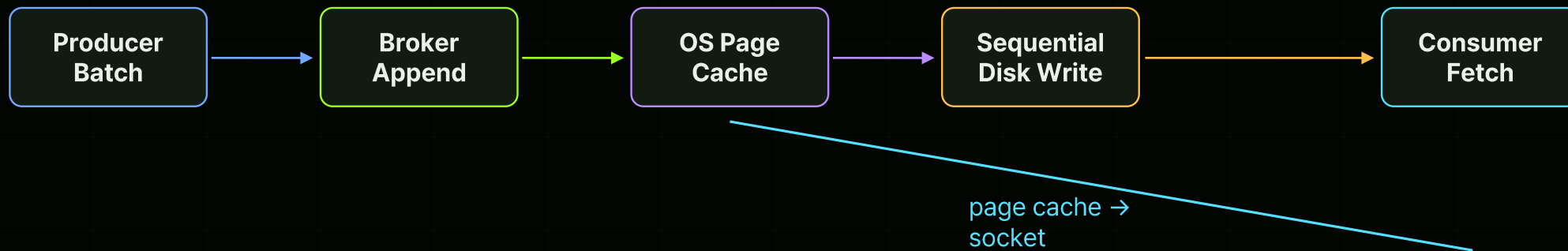
timestamp 기반 탐색과
retention을 돕습니다.

Retention

시간/용량 기준으로 오래된
segment를 삭제합니다.

왜 디스크인데 빠른가?

핵심은 random I/O를 피하고, OS가 잘하는 sequential I/O와 page cache를 활용하는 것입니다.



Sequential access

디스크는 임의 위치를 자주 찾는 것보다 연속 영역을 읽고 쓰는 작업에 강합니다.

Page cache

최근에 쓰고 읽은 log segment는 OS cache에 남아 consumer fetch를 빠르게 합니다.

Zero-copy

가능하면 user space 복사를 줄이고 cache에서 network socket으로 바로 보냅니다.

Batching

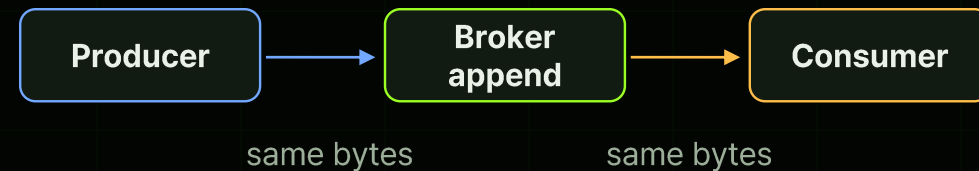
작은 메시지를 모아 요청 수와 system call 수를 줄입니다.

메시지 자료구조와 일괄 처리

메시지를 중간에서 수정하지 않도록 설계하면 producer → broker → consumer 경로가 단순해집니다.

Record Batch

Batch Header	baseOffset, timestamp, codec, crc
Record 0	key, value, headers
Record 1	key, value, headers
Record 2	key, value, headers
Record N	key, value, headers

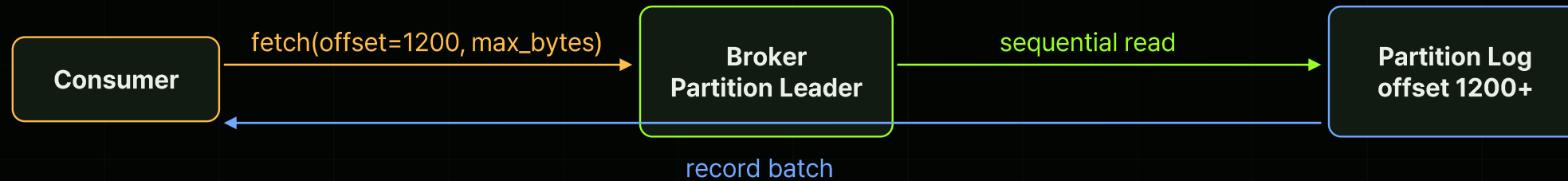


Batch의 효과

- 여러 메시지를 하나의 produce request로 전송
- 브로커는 batch 단위로 append하고 checksum 확인
- 압축은 record별보다 batch별로 효율이 좋음
- 큰 batch는 throughput을 높이지만 latency를 늘림

읽기 경로: Pull 모델

Consumer가 자신의 속도에 맞춰 broker에서 데이터를 가져옵니다.



소비 속도 제어

느린 consumer가 broker에게 무한 push를 받지 않습니다.

Aggressive batching

consumer가 충분한 byte 수가 찰 때까지 기다리게 할 수 있습니다.

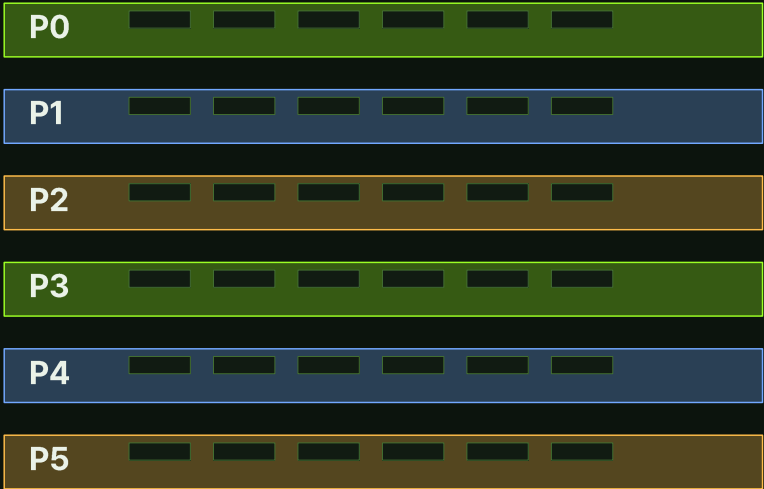
Replay

offset을 되돌리면 보관 기간 안의 메시지를 다시 읽을 수 있습니다.

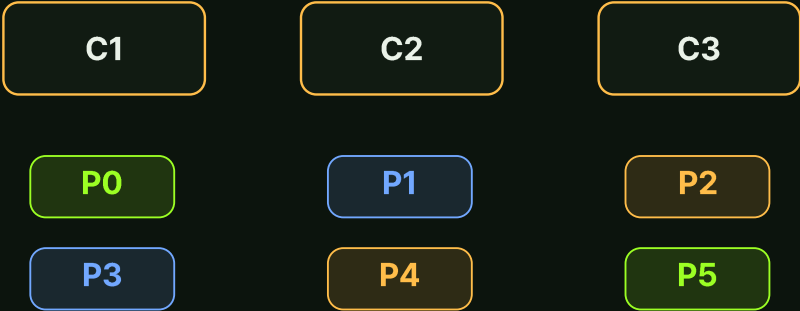
Consumer Group과 파티션 할당

같은 group 안에서 하나의 partition은 한 consumer에게만 할당됩니다.

Topic: order-events



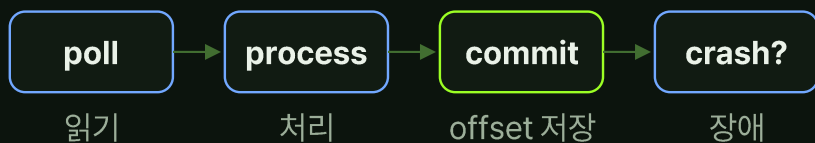
Consumer Group: shipment-service



Offset Commit과 재처리

offset commit 시점이 메시지 유실과 중복의 경계를 만듭니다.

처리 후 commit: at-least-once



- 처리 성공 후 offset을 commit합니다.
- 장애가 commit 전에 나면 같은 메시지를 다시 읽을 수 있습니다.
- 유실보다 중복을 감수하는 기본 선택입니다.

처리 전 commit: at-most-once



- 처리 전에 offset을 먼저 commit합니다.
- 장애가 process 전에 나면 메시지는 건너뛴 것으로 보입니다.
- 중복은 줄지만 유실 가능성이 생깁니다.

전달 보장 방식

보장 수준이 올라갈수록 구현 복잡도와 지연 비용도 함께 올라갑니다.



At-most-once

0 또는 1회 전달
중복 없음, 유실 가능



At-least-once

1회 이상 전달
유실 없음, 중복 가능



Exactly-once

정확히 1회처럼 처리
트랜잭션/멱등성 필요

선택 기준

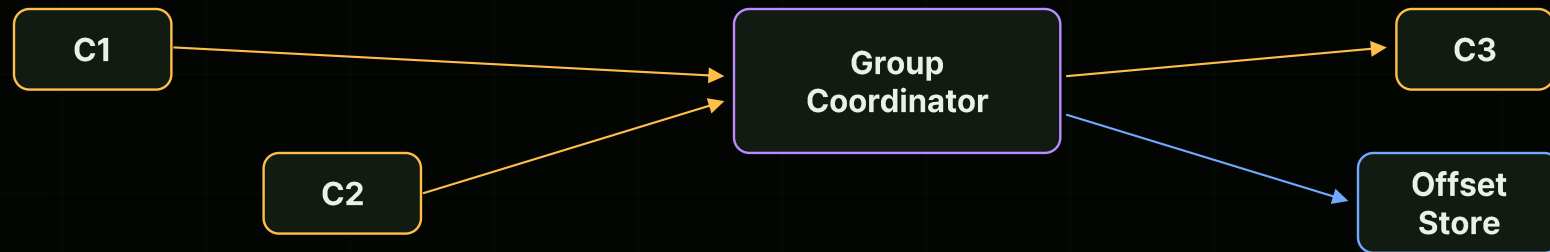
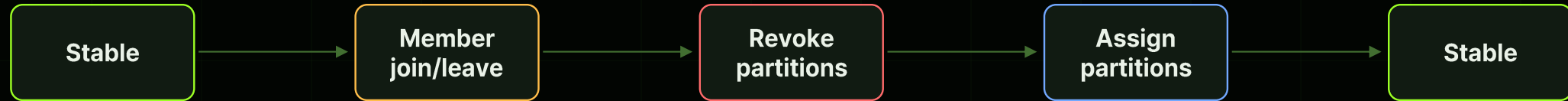
로그·통계 이벤트 → at-most/at-least를 요구에 맞게 선택

주문·결제 이벤트 → at-least-once + 멱등 처리 또는 transaction 패턴

정확히 한 번 “처리 결과”까지 보장하려면 외부 저장소와 offset의 원자성이 핵심

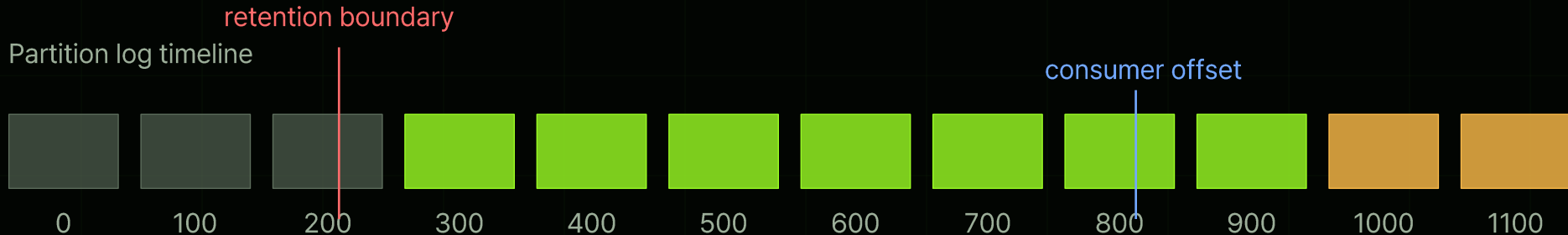
Rebalancing과 Group Coordinator

Consumer가 들어오거나 나가면 partition ownership을 다시 나눕니다.



장기 보관과 반복 소비

큐에서 읽었다고 메시지를 바로 삭제하지 않으면, 여러 시스템이 같은 데이터를 독립적으로 재처리할 수 있습니다.



Replay

새로운 소비자 그룹이 과거 데이터를 처음부터 읽어 모델/검색 인덱스를 다시 만들 수 있습니다.

Backfill

버그 수정 후 특정 offset 구간만 다시 처리할 수 있습니다.

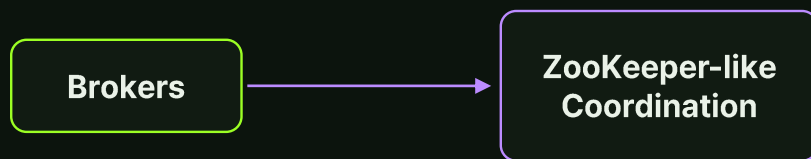
Audit

이벤트 원본을 일정 기간 보관하여 추적 가능성을 높입니다.

Metadata와 Coordination

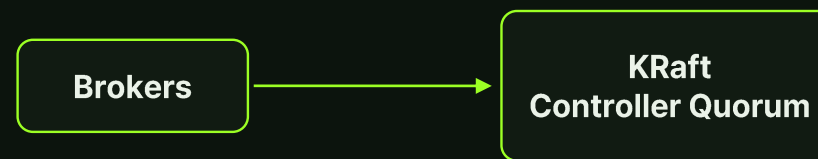
데이터 저장소와 제어 메타데이터를 분리해야 broker cluster를 안정적으로 운영할 수 있습니다.

책의 설명에 가까운 구조



- broker liveness
- topic/partition metadata
- controller election
- consumer group coordination

현대 Kafka에서 볼 수 있는 구조



- metadata를 Kafka 내부 quorum에서 관리
- broker와 controller 역할 분리
- ZooKeeper 의존성 축소/제거
- 운영 단순화와 장애 격리 기대

확장성 설계: Partition이 곧 병렬성

Broker를 늘리는 것만으로 충분하지 않습니다. Topic partition 설계가 consumer 병렬성을 결정합니다.

Partition 수



Consumer 병렬성

파티션 증가의 장점

- 쓰기/읽기 병렬성 증가
- broker 간 저장량 분산
- consumer group 처리량 증가

파티션 증가의 비용

- metadata와 file handle 증가
- rebalance 비용 증가
- key별 순서 범위가 좁아짐

Hot key 대응

- partition key 재설계
- key salting / sub-key
- producer quota와 backpressure

신뢰성과 장애 대응

장애 대응은 producer, broker, consumer 세 경로를 따로 봐야 합니다.

Producer 장애

retry + idempotence

중복 가능성을 producer id/sequence로 제어

delivery timeout과 backoff 설정

Broker 장애

leader election

ISR replica 승격

under-replicated partition 감시

Consumer 장애

heartbeat timeout

rebalance 후 다른 consumer가 이어 처리

마지막 committed offset부터 재시작

Downstream 장애

처리 실패 재시도

poison message 격리

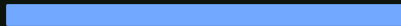
DLQ/parking lot topic 고려

운영 지표와 관측성

메시지 큐 운영에서 “정상처럼 보이지만 뒤쳐지는 상태”를 빨리 발견해야 합니다.

Producer

- produce latency
- error/retry rate
- batch size
- compression ratio



Broker

- request queue time
- disk usage
- network IO
- under-replicated partitions



Consumer

- consumer lag
- poll latency
- commit latency
- rebalance count



Cluster

- active controller
- ISR shrink/expand
- partition skew
- quota throttle time



대표 알람 예시

consumer lag가 계속 증가한다면 downstream 처리량 부족, hot partition, rebalance 폭주, broker fetch 지연 중 하나를 의심합니다.

트레이드오프와 면접 답변 포인트

좋은 답변은 “무엇을 선택했는가”보다 “무엇을 포기했는가”를 설명합니다.

Push vs Pull

Pull 선택: consumer backpressure와
batch fetch가 쉬움 / 대신 즉시
push보다 latency가 늘 수 있음

Batch size vs Latency

큰 batch : throughput 상승
작은 batch : tail latency 개선

ACK level vs Durability

acks=all : committed data 보호
acks=0/1 : 낮은 지연 대신 유실 가능성

Partition count vs Ordering

partition 증가 : 병렬성 상승
전체 순서 보장은 어려워짐

Retention vs Cost

장기 보관 : replay/backfill 가능
저장 비용과 복제 비용 증가

핵심 요약

분산 메시지 큐 설계는 여섯 가지 선택으로 압축할 수 있습니다.

1. Topic / Partition

논리적 주제와 물리적 확장 단위를 분리합니다.

2. Append-only log

메시지는 partition log 끝에 순차 기록합니다.

3. Batch + Page cache

네트워크와 디스크 비용을 batch로 줄입니다.

4. Consumer Group

그룹 안에서는 분산 처리,
그룹 간에는 독립 소비합니다.

5. Offset Commit

재처리, 중복, 유실의 경계를 결정합니다.

6. Replication + Rebalance

장애 후에도 읽기/쓰기를 이어가도록 합니다.

스터디 토론 질문

설계 선택을 바꾸면 어떤 트레이드오프가 생기는지 함께 토론해 봅시다.

Q1

결제 이벤트 큐에서 “중복 처리”와 “이벤트 유실” 중 하나만 피할 수 있다면 무엇을 우선해야 할까요? 그 이유는 무엇인가요?

Q2

전체 메시지 순서 보장을 요구하는 기능이 들어오면 partition 설계를 어떻게 바꿀까요? 처리량 손실을 줄일 방법은 있을까요?

Q3

Consumer lag가 갑자기 증가했을 때 producer, broker, consumer, downstream 중 어디부터 확인해야 할까요?